

# Assignment-3

## Introduction

This project aims to simulate a preemptive process scheduler and a simple command-line shell in the C programming language.

The project demonstrates key Operating System concepts such as process creation, inter-process communication, signal handling, and round-robin CPU scheduling.

The system consists of two separate programs:

- SimpleShell – acts as the user interface, allowing users to submit and manage processes.
- SimpleScheduler – operates as a background daemon, responsible for allocating CPU time slices to submitted processes in a fair, round-robin fashion.

The objective is to show how a real OS scheduler switches between multiple processes, ensuring each process gets CPU time in a preemptive multitasking environment.

## Approach

The design follows a modular approach, separating the shell (user interaction) and the scheduler (process management):

### **a. SimpleShell**

- Takes NCPU (number of CPU cores) and TSLICE (time slice in milliseconds) as command-line inputs.
- Displays a prompt where the user can type commands like:  
SimpleShell\$ submit ./a.out
- On submit, the shell:
  - a. Creates a new child process using fork().
  - b. Suspends it immediately with raise(SIGSTOP) so that it doesn't run until the scheduler allows.
  - c. Sends the process PID to the scheduler through a pipe.
- Handles SIGCHLD to track job completion.
- On exit, closes the pipe and prints a report (collected from the scheduler).

### **b. SimpleScheduler**

- Launched automatically by the shell using execvp.
- Reads process PIDs from the pipe and adds them to a ready queue.
- Uses a timer interrupt (setitimer() with SIGALRM) to trigger scheduling every TSLICE milliseconds.
- At each tick:
  - Stops currently running processes (SIGSTOP).
  - Moves them to the rear of the queue.
  - Starts the next set of processes (SIGCONT) according to available CPU slots (NCPU).
- Maintains statistics such as executed and waited time slices for each process.
- On completion (after all jobs finish naturally), it writes a report file (/tmp/scheduler\_report.dat) which the shell later reads.

# AI usage

Some implementation and fixes for preemptive multitasking and round-robin scheduling (using SIGSTOP, SIGCONT, and periodic SIGALRM signals) -schedule function that first stops running processes and requeues them, then dispatches new processes from the ready queue, and finally updates the execution and wait time statistics for all tracked jobs.

The configuration of the timer structure (struct itimerval and setitimer()) for generating periodic interrupts was used from AI:

Set up the timer for preemptive multitasking

```
timer.it_value.tv_sec = TSLICE / 1000;  
timer.it_value.tv_usec = (TSLICE % 1000) * 1000;  
timer.it_interval = timer.it_value;
```

Using the temporary file path /tmp/scheduler\_report.dat follows the Unix convention for short-term data storage and inter-process reporting.

Handling SIGCHLD to detect completed jobs.

kill(pid, SIGSTOP) / kill(pid, SIGCONT) / kill(pid, SIGTERM) - Used for preemptive control of child process.

kill(pid,0) - to check either process is alive or not .

SIGKILL - handle termination

Raise(SIGSTOP)- Pause itself immediately

waitpid(-1, &status, WNOHANG)-The use of a non-blocking waitpid() in a while loop to reliably prevent zombie processes.

Logic for terminate scheduler process using kill stop and close before shell exits.

## Workflow

Below is the overall system workflow from start to termination:

### 1. Initialization

- The user runs ./SimpleShell NCPU TSLICE.
- SimpleShell creates a pipe and forks a SimpleScheduler process.

### 2. Job Submission

- User submits a job using submit ./a.out.
- Shell forks a new process, suspends it, and sends its PID through the pipe to the scheduler.

### 3. Scheduling

- Scheduler reads PIDs and adds them to the ready queue.
- The timer sends a SIGALRM signal every TSLICE ms.
- On each signal:
  - Running processes are paused (SIGSTOP).
  - Next jobs from the queue are resumed (SIGCONT).

#### 4. Process Execution

- Processes execute in fixed-length time slices in a round-robin manner.
- Statistics for each process are recorded (execution time slices, waiting time slices).

#### 5. Termination

- When the user types exit, the shell closes its pipe.
- Scheduler detects EOF, waits until all jobs naturally finish, then writes the report.
- Shell reads /tmp/scheduler\_report.dat and displays:
  - 
  - Command PID Completion Time Wait Time
- Both programs exit cleanly.

## Contributions

|              |         |                                                                                                                                                                                                                                                                     |
|--------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Somya        | 2024560 | Implemented the SimpleShell program including user input handling, process creation with fork() and execvp(), signal handling (SIGCHLD), and final job report generation. Designed the command parsing logic and integrated inter-process communication using pipes |
| Arjun Khatri | 2024106 | Implemented the SimpleScheduler program, including the ready queue, timer setup using setitimer(), signal-based preemptive scheduling (SIGALRM, SIGSTOP, SIGCONT), and job statistics management. Also handled report file creation and cleanup logic.              |

# **References**

Lecture 06-13

## **Github Link**

Github Link